

Sub phase-5 Performance Testing

Date	28-09-2026
Team ID	SWTID-2026-7624
Project name	Script-controlled ACL
Maximum marks	4 mark

PERFORMANCE TESTING

Introduction

Performance Testing is a type of software testing used to evaluate the speed, responsiveness, stability, scalability, and reliability of a software application under different workloads. The main purpose of performance testing is to determine whether an application can provide acceptable performance when it is used by a normal or large number of users.

Performance testing helps identify performance-related issues before the application is released to actual users. It evaluates how quickly the system responds to user requests and how efficiently it uses available system resources.

Objectives of Performance Testing

The major objectives of performance testing are:

- To measure the response time of the application.
- To check the speed and responsiveness of the system.
- To identify performance bottlenecks.
- To verify system stability under different workloads.
- To determine the maximum workload that the application can handle.
- To evaluate the scalability of the application.
- To check CPU, memory, database, and network resource utilization.
- To ensure that the application provides consistent performance to users.

Important Performance Parameters

1. Response Time

Response time is the amount of time taken by the application to respond to a user's request. Lower response time generally indicates that the application is responding quickly.

For example, when a user opens a dashboard or submits a form, the time taken by the system to display the result can be measured.

2. Throughput

Throughput refers to the number of requests or transactions that a system can process within a specific period of time. Higher throughput indicates that the system can handle more operations efficiently.

3. Scalability

Scalability refers to the ability of an application to maintain acceptable performance when the workload, data volume, or number of users increases.

A scalable system should continue to work efficiently even when additional users or requests are introduced.

4. Resource Utilization

Performance testing also monitors system resources such as:

- CPU usage
- Memory usage
- Database usage
- Network usage
- Disk usage

High resource consumption may indicate a performance bottleneck that needs to be investigated.

5. Stability

Stability refers to the ability of the application to operate continuously without crashes, failures, or significant performance degradation.

Long-duration testing can be performed to determine whether the application remains stable over an extended period.

Types of Performance Testing

1. Load Testing

Load testing checks how the application behaves under an expected number of users or workload.

The objective is to verify whether the system can handle normal business usage while maintaining acceptable response time and stability.

2. Stress Testing

Stress testing evaluates the application beyond its normal operating capacity. The workload is gradually increased to identify the point at which the system starts experiencing performance problems.

It helps determine the limitations of the application.

3. Volume Testing

Volume testing checks how the application performs when processing a large amount of data.

For example, a database containing a large number of records can be used to determine whether queries and reports continue to perform efficiently.

4. Endurance Testing

Endurance testing, also called soak testing, checks whether the application can operate continuously for a long period.

It helps identify memory leaks, resource exhaustion, and gradual performance degradation.

5. Spike Testing

Spike testing evaluates how the system behaves when there is a sudden and significant increase or decrease in workload.

It is useful for applications where user traffic can change unexpectedly.

Performance Testing Process

Step 1: Identify Requirements

The performance requirements of the application are identified. These may include expected response time, number of users, transaction volume, and acceptable resource utilization.

Step 2: Prepare the Test Environment

A suitable testing environment is prepared with the required application, database, network, hardware, and testing tools.

Step 3: Create Test Scenarios

Different real-world scenarios are prepared. These scenarios represent the activities that users are expected to perform.

Step 4: Execute Performance Tests

The application is tested under different workloads. The number of users or transactions can be increased to observe system behaviour.

Step 5: Monitor Performance

Important parameters such as response time, throughput, CPU utilization, memory usage, and error rate are monitored during testing.

Step 6: Analyze Results

The collected performance results are analyzed to identify slow operations, bottlenecks, errors, or resource-related problems.

Step 7: Optimization

If performance problems are identified, the application or database configuration can be optimized. The test can then be repeated to verify the improvement.

Advantages of Performance Testing

Performance testing provides several benefits:

- Improves application responsiveness.
- Helps identify performance bottlenecks.
- Improves system stability.
- Helps determine system capacity.
- Supports scalability planning.
- Reduces the possibility of performance failures after deployment.
- Improves overall user experience.
- Helps optimize system resources.

Performance Testing in Real-World Applications

Performance testing is important for applications such as banking systems, e-commerce websites, cloud applications, business intelligence dashboards, CRM platforms, and database-driven applications.

For example, a business dashboard may need to display information quickly even when thousands of records are available. Performance testing can determine whether the dashboard, database queries, and visualizations continue to respond efficiently under different workloads.

Expected Result

After successful performance testing, the application should provide acceptable response time, stable operation, efficient resource utilization, and reliable performance under the expected workload.

Conclusion

Performance Testing is an important part of software testing that ensures an application can perform efficiently under different workloads. It helps measure response time, throughput, scalability, stability, and resource utilization. By identifying and resolving performance bottlenecks before deployment, performance testing helps deliver a more reliable and responsive application to end users.